RAPPORT DE TRAVAUX PRATIQUES

المدرسة الوطنية للعلوم التطبيقية - بني ملال

Ecole Nationale des Sciences Appliquées - Béni Mellal

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BÉNI MELLAL

TP2 – Monolithe Modulaire

Réalisé par : EL MELIANI Ammar
Encadrant : Pr. EL BAGHAZAOUI

Filière : **IACS-S2**
Année universitaire : 2025/2026

TP 2 - Monolithe Modulaire

1. Objectifs du TP

- **Restructurer le monolithe en modules métier distincts**
- **Définir des boundariès claires entre domaines**
- **Implémenter les design patterns DTO, Mapper et Service Layer**
- **Utiliser MapStruct pour le mapping automatique Entity <-> DTO**
- **Préparer le terrain pour une future migration vers les microservices**

2. Nouveaux Concepts Introduits

2.1 DTO (Data Transfer Object)

Un DTO est un objet intermediaire utilise pour transferer des donnees entre les couches de l'application. Au lieu d'exposer directement l'entite JPA au client, on cree un DTO qui contient uniquement les champs necessaires. Cela apporte plusieurs avantages :

- **Securite :** on ne expose pas les details internes de la base de donnees
- **Flexibilite** : le DTO peut avoir une structure differente de l'entite
- **Versioning** : on peut modifier l'entite sans casser l'API publique
- Performance : on peut limiter les donnees transferees sur le reseau

2.2 MapStruct

MapStruct est un generateur de code Java qui cree automatiquement le code de conversion entre les entites et les DTOs a la compilation. Il elimine le code repetitif de mapping manuel et offre de meilleures performances que les mappers par reflexion comme ModelMapper.

2.3 Interface + Implementation (Service Layer Pattern)

Le Service Layer Pattern separe le contrat (interface) de l'implementation. Cela permet de :

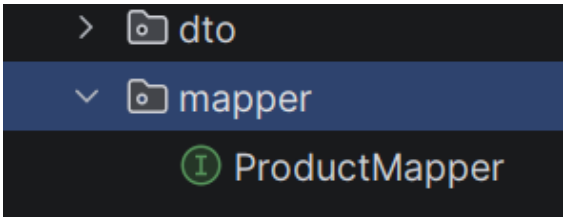
- Faciliter les tests unitaires (on peut mocker l'interface)
- Respecter le principe SOLID (Open/Closed Principle)
- Faciliter le remplacement de l'implementation sans changer le code appelant

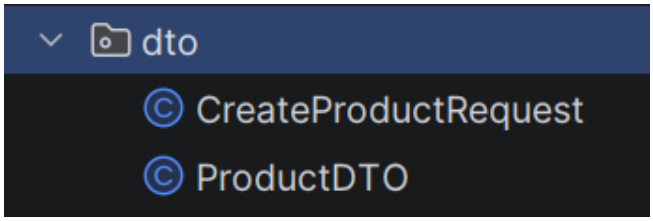
3. Nouvelle Architecture

La structure du projet est reorganisee pour mieux separer les responsabilites :

```
com.ecommerce.monolith
  product/
```

```
controller/   -> ProductController.java
service/      -> ProductService.java (interface)
              -> ProductServiceImpl.java (implementation)
repository/   -> ProductRepository.java
model/        -> Product.java
dto/          -> ProductDTO.java
              -> CreateProductRequest.java
mapper/       -> ProductMapper.java
```





4. Implementation des DTOs

4.1 ProductDTO

Le ProductDTO est utilise pour les reponses de l'API. Il contient tous les champs du produit qui doivent etre retournes au client.

4.2 CreateProductRequest

Le CreateProductRequest est utilise pour les requetes de creation et de modification. Il contient les validations necessaires et n'expose pas le champ id (genere automatiquement).

5. Refactorisation du Service

Le service est separe en deux fichiers : l'interface ProductService qui definit le contrat, et ProductServiceImpl qui contient l'implementation concrete. Le mapper est injecte dans le service via l'injection de dependances de Spring.

Classe/Interface	Role
ProductService (interface)	Definit les methodes disponibles - contrat public
ProductServiceImpl (classe)	Implementation concrete avec logique metier
ProductMapper (interface)	Conversion automatique Entity <-> DTO par MapStruct
ProductDTO	Objet de reponse envoye au client
CreateProductRequest	Objet de requete recu du client avec validation

6. Configuration MapStruct

MapStruct est ajoute dans build.gradle avec trois dependances : la librairie principale, le processeur d'annotations qui genere le code, et le binding avec Lombok pour assurer la compatibilite.

7. Flux d'une Requete dans le TP2

Le flux complet d'une requete POST /api/products dans le TP2 est le suivant :

Etape	Composant	Action
1	Client (Postman)	Envoie POST /api/products avec JSON CreateProductRequest
2	ProductController	Recoit la requete, valide (@Valid), appelle productService.createProduct()
3	ProductServiceImpl	Recoit CreateProductRequest, appelle mapper.toEntity()
4	ProductMapper	Convertit CreateProductRequest -> Product (genere par MapStruct)
5	ProductRepository	Sauvegarde le Product dans PostgreSQL via repository.save()
6	ProductServiceImpl	Recoit le Product sauvegarde, appelle mapper.toDTO()
7	ProductMapper	Convertit Product -> ProductDTO
8	ProductController	Retourne ResponseEntity avec ProductDTO et statut 201 Created
9	Client (Postman)	Recoit la réponse JSON avec le ProductDTO

8. Conclusion TP2 :

- **Le TP2 a introduit les concepts professionnels du développement Spring Boot : le pattern DTO pour la sécurité et la fléxibilite de l'API, MapStruct pour le mapping automatique, et le Service Layer Pattern pour la testabilité et la maintenabilité.**
- **La restructuration en modules distincts avec des boundaries claires est la base de toute architecture logicielle professionnelle. Cette organisation facilite le travail en équipe, les tests unitaires, et prépare directement la migration vers les microservices.**

Fin